CleanROMs

Complete User Manual — Version 2.8

CleanROMs is a command-line (PowerShell) tool for detecting duplicate ROMs inside a RetroBat installation, automatically keeping the best copy of each game, moving the rest to a backup folder (it never deletes them on its own; only an optional purge that you start yourself removes old copies, and it asks first), and also cleaning up orphaned images, videos, and manuals left behind by ROMs that were already removed.

Development of this program started with ChatGPT (GPT-5.5) and was completed and debugged with Claude, Anthropic's AI assistant, through several rounds of real-world testing on large collections (several thousand ROMs per system).

Repository: https://github.com/batserra/CleanRoms

Table of Contents

• What's new in version 2.8
• 1. What this program does
• 2. How it decides which ROM to keep: the scoring system
• 3. When there's a tie: tie-break order
• 4. Practical step-by-step example
• 5. What's always excluded (Hacks, Betas, Demos, Homebrew...)
• 6. How it recognizes that two ROMs are the same game
• 7. Title aliases: when the name is genuinely different
• 8. Associated files: save files and configurations
• 9. Cleaning up orphaned images, videos, and manuals
• 10. Requirements and installation
• 11. First run: RetroBat, language, and ROM priority
• 12. Configuring the program in depth
• 13. The main menu (the 7 options)
• 14. Simulation mode (PreviewOnly)

• 15. Generated reports and the full log
• 16. Undoing the last cleanup
• 17. Supported systems and extensions
• 18. Special file names (brackets, apostrophes, underscores)
• 19. Interface language and ROM priority
• 20. About renaming ROMs (RENAME)
• 21. Command-line use: automated/unattended runs
• 22. Tests performed
• 23. Multi-file games (.cue/.bin, .gdi, multi-disc, .m3u)
• 24. Manual decisions that stick (Overrides.json)
• 25. RetroBat's gamelist.xml (favorites, play counts...)
• 26. Hash cache and faster deduplication
• 27. Purging _duplicates (menu option 6)
• 28. Systems and extensions in JSON files
• 29. Version number and releases
• 30. Reporting issues

What's new in version 2.8

This version adds the features below. Each one has its own section in this manual, shown in parentheses.

• Multi-file games (.cue/.bin, .gdi, several discs, .m3u) are handled as one unit, so a game is never split in two (section 23).

• "Undo the last cleanup" also restores the orphaned images, videos, and manuals, undoes moves from last to first, and keeps going if one file cannot be restored (section 16).

• RetroBat's gamelist.xml: favorites, play counts, and ratings follow the ROM that stays, and the entries of hacks follow the files when they are organized into "# Hacks y Otros #" (section 25).

• Manual decisions that stick, in Config\Overrides.json, and the program offers to remember your tie-break answers (section 24).

• Hash cache: files that have not changed are not hashed again (section 26).

• Purge of _duplicates, menu option 6 or -Action Purge: the only thing that deletes files, always after showing the list and asking (section 27). The menu now has 7 options (section 13).

• Systems, extensions, and ignored folders moved to Config\Systems.json and Config\Extensions.json, with optional .custom.json files for your own additions (section 28).

• Exit codes for scheduled tasks, and no waiting for a key at the end in unattended mode (section 21).

• One version file (VERSION), a tool and a GitHub workflow to build and publish the release ZIP, and every on-screen message now in Spanish and English (section 29).

• Automated tests and continuous integration (section 22).

1. What this program does

CleanROMs scans the ROM folders of your RetroBat systems and groups together the copies that correspond to the same game (for example, the USA version, the European version, and a compressed copy of the same title). For each group, it scores every copy according to a set of criteria (region, language, dump quality, version...) and decides which one is best. The rest are moved — not deleted — to a backup folder (the only thing that ever deletes files is the optional purge in section 27).

In summary, on every run:

1 It scans the chosen folder (or folders), looking for files with a recognized ROM extension.

2 It parses each file name (and, if it's a .zip/.7z containing a single ROM, the inner file name too) to extract region, language, version, revision, and special flags.

3 It normalizes each ROM's title (see section 6) and groups the ones that correspond to the same game, ignoring those tags — and applying manual aliases for cases where the name genuinely changes from one language to another (see section 7).

4 It scores every copy within its group and picks the one with the highest score.

5 It shows you the full plan (what's kept, what's moved, and why) before touching anything.

6 Only if you confirm with "Y" (or "yes"), it moves the non-chosen copies to _duplicates\ — along with any associated file they had (save file, controller configuration...), see section 8.

Besides cleaning up ROMs, it can also detect orphaned images, videos, and manuals (from ScreenScraper/RetroBat) that no longer correspond to any existing ROM, undo the last cleanup you ran, and do everything at once for your entire collection. All of this is explained further below.

2. How it decides which ROM to keep: the scoring system

Each copy of a game receives a numeric score, adding up the points for every criterion it meets. The copy with the highest score in the group is the one that's kept ("KEEP"); the rest are moved ("MOVE"). These are the exact point tables the program uses (they live in Config\DecisionWeights.ps1, and can be freely adjusted if you want to change the priorities).

Region and language: two tables, depending on your priority

As of version 2.7, region and language have **two** possible point tables, not just one: one for when you prefer Spanish ROMs, and another for when you prefer English ROMs. Which one is active depends on the "ROM priority" setting, which is asked the first time you run the program and can be changed any time from menu option 5 (see sections 11 and 19). The rest of the tables in this section (dump quality, status, hacks, version, revision) are always the same, they don't depend on this setting.

Region	Points with SPANISH priority	Points with ENGLISH priority
Spain (ESP)	1000	400

Region	Points with SPANISH priority	Points with ENGLISH priority
Europe (EUR)	700	700
USA	400	1000
Japan (JPN)	200	200
World	100	100
Unknown	0	0

With Spanish priority, a Spanish copy will almost always beat any other combination, unless it's also a bad dump, a hack, etc. (see section 5). With English priority the exact same thing happens but with USA/English — the rest of the positions (Europe, Japan, World) don't change places.

Language

Language	Points with SPANISH priority	Points with ENGLISH priority
Spanish (single language)	500	100
Multi-Spanish (includes Spanish among several)	350	200
Multi (several languages, no Spanish)	200	350
English	100	500
Japanese / Unknown	0	0

Dump quality

Criterion	Points
Verified [!] (confirmed good dump)	+200
Bad dump [b]	-500

ROM status

Status	Points
Beta	-150
Prototype	-300
Demo	-300
Sample	-300
Preview	-300
Kiosk	-300

> *These are all penalties: a Beta or Prototype version almost never wins over the final release, even if it has a better region or language.*

Hacks, Homebrew, and Pirate

Type	Points
Hack	-400
Homebrew	-400
Pirate	-500

> *Note: this penalty would only apply if one of these ROMs ever got compared against another — but as explained in section 5, Hacks, Homebrew, Pirate, Betas, Prototypes, Demos, Samples, Previews, and Kiosks are excluded from comparison entirely, so in practice this score is almost never actually used. It's documented here in case you ever decide to enable comparison for them.*

Version and revision

Version	Points
V1.0	100
V1.1	110
V1.2	120
V1.3	130
No version given	0

Revision	Points
Rev 0	0
Rev 1	10
Rev 2	20

Revision	Points
Rev 3	30
Rev 4	40
No revision given	0

> *These are the two lowest-weighted criteria: they mainly serve to decide between two copies that already share region, language, and status — the higher the version or revision number, the more points (on the assumption that it fixes bugs from earlier ones). A version not listed in the table (e.g. V1.4) is calculated on its own using the same scale, instead of giving 0 points.*

3. When there's a tie: tie-break order

If two copies end up with the same total score, they're compared in this exact order until something tells them apart:

1 Total score (explained in section 2).

2 Is one verified [!] and the other not? The verified one wins.

3 Is one a bad dump [b] and the other not? The bad dump loses.

4 Is one a Hack and the other not? The Hack loses.

5 Is one a Prototype and the other not? The prototype loses.

6 Is one a Beta and the other not? The beta loses.

7 Is one a Demo and the other not? The demo loses.

8 Is one compressed (.zip/.7z) and the other not? The one that is NOT compressed wins.

9 Do the titles have different lengths? The shorter title wins.

10 Are the titles alphabetically different? The one that comes first alphabetically wins.

11 If after all of this they're still exactly tied (same score, same criteria, same name): the program asks you which one you'd rather keep, showing you both full paths.

This last step (asking you) only happens when there truly is no automatic criterion left to tell them apart — in a normal collection this is uncommon, because step 8 (preferring the uncompressed one) already resolves the most typical case (the same ROM as a .zip and uncompressed). In non-interactive mode (see section 21), this question answers itself by choosing option 1, without blocking execution.

4. Practical step-by-step example

Imagine you have these three copies of the same game in your SNES folder:

• A) Game (Europe) (En,Fr,De,Es).sfc

• B) Game (U) (V1.1) [!].sfc

• C) Game [E].zip

The program would group them like this (all three are "the same game" once the name is normalized), and score them (with Spanish priority, the default option):

Criterion	A) Europe/Multi-Es	B) USA V1.1 Verified	C) [E] compressed
Region	EUR = 700	USA = 400	EUR = 700
Language	Multi-Spanish = 350	Unknown = 0	Unknown = 0
Verified	No = 0	Yes = +200	No = 0
Version	— = 0	V1.1 = +110	— = 0
TOTAL	1050	710	700

Result: copy A is kept (1050 points, winning mainly because it includes Spanish among its languages), and B and C are moved to _duplicates\snes — along with any save file or controller configuration they had associated (see section 8).

5. What's always excluded (Hacks, Betas, Demos, Homebrew...)

There are nine categories of ROMs that the program never compares, never moves, and never renames, no matter what their score is:

Category	How it's detected
Hack	The word "Hack" in the name (as a standalone word, surrounded by spaces, parentheses, or underscores — not inside another word like "Protocol"), or being inside a folder like "# Hacks y Otros #"
Fan translation	"Traducción"/"Translation"/"Trans", [T+Spa] tags or T(ESP)/T(POR), or a "Traducciones" folder
Beta	The word "Beta" in the name, as a standalone word
Prototype	"Proto", "Prototype", or "Prototipo" in the name, as a standalone word
Demo	The word "Demo" in the name, as a standalone word
Homebrew	The word "Homebrew" in the name, as a standalone word
Pirate	The word "Pirate" in the name, as a standalone word
Sample	The word "Sample" in the name, as a standalone word
Preview / Kiosk	"Preview" or "Kiosk" in the name, as a standalone word

The reason for excluding Beta/Prototype/Demo/Homebrew/Pirate/Sample/Preview/Kiosk, on top of Hacks and Translations, is that they represent content that's genuinely different from the final/retail release — a

beta or a prototype isn't "the same ROM with a worse name", it's a different version a collector may want to keep separately. Merging it with the final release would risk losing something unique if the decision engine chose to move it thinking it was just a duplicate.

Standalone words only, not substrings

All of these checks require the word to appear as a **standalone** word — bounded by something that isn't a letter (space, parenthesis, underscore, start/end of the name...), never as part of a longer word. This matters: without this check, completely legitimate games like *Demon's Crest*, *Pirates of the Caribbean*, or *Team Protoman* would be misclassified as a demo/pirate/prototype, just because those letters appear inside a different word ("Demo" inside "Demon", "Pirate" inside "Pirates", "Proto" inside "Protoman").

> *One important nuance: if you have two copies of the same Beta, the same Hack, etc. (for example, a .smc and a .zip of the same hack), they also won't be detected as duplicates of each other by title — they're kept completely outside the normal cleanup system. There's a separate mechanism for that specific case: see "organizing loose hacks" and "deduplicating by hash" in section 13, option 1.*

6. How it recognizes that two ROMs are the same game

Before comparing anything, the program turns every file name into a "normalized title": a clean, lowercase version with nothing left except the game's actual name. Two ROMs are considered the same game if their normalized title is identical. These are, in order, the cleanup steps it applies:

1 Leading catalog number: strips a prefix like "0263 - " at the start of the name, typical of some numbered sets.

2 Date/time suffix: strips marks like "_20260709_034928" that some copying tools add when renaming duplicates.

3 Underscore-style names: some sets (mostly SNES) replace spaces, apostrophes, and parentheses with "_". The program recognizes this pattern: it converts the encoded apostrophe ("Kirby_s_Fun_Pak" →

"Kirbys Fun Pak"), strips the trailing tags (region, version, hack, translation credit) joined by underscores, and only then converts the remaining underscores into spaces — so words like "and"/"the"/"el"/"la" are detected just as well as in a normal name.

4 Parentheses and brackets: ALL of their content is removed, whatever it is (year, region, language, publisher, dump group, revision...). In the vast majority of ROM sets (No-Intro, TOSEC, GoodTools...) everything between () or [] after the title is metadata, never part of the game's name.

5 Loose region/language words: in case they appear without parentheses (rare, but kept as a safety net): spanish, english, french, german, italian, portuguese, dutch, multi, usa, europe, japan, world, etc.

6 Revision and version: a loose "Rev A", "V1.1" (outside parentheses) is also stripped, always requiring a clear separator so it doesn't get confused with real words that start the same way (for example, this keeps "Revolver" or "Twin Turbo V8" intact).

7 Roman numerals: II, III, IV, V, VI, VII, VIII are converted to digits (2, 3, 4, 5, 6, 7, 8) so that "Final Fantasy IV" and "Final Fantasy 4" are recognized as the same title.

8 Articles: "the"/"a"/"an" in English, and "el"/"la"/"los"/"las" in Spanish, are stripped (so "The Lord of the Rings" and "Lord of the Rings, The" match, just like "Los Sims 2" and "Sims 2, The").

9 Apostrophes, periods, and exclamation/question marks: "Hoodlum's" = "Hoodlums", "Jr." = "Jr", "Day of Discovery!" = "Day of Discovery".

10 "&", "+", "and", "y": all treated as equivalent, so "Tom & Jerry" = "Tom and Jerry", and "Rayman Advance & Rayman 3" = "...+ Rayman 3".

11 Remaining symbols (dashes, colons, commas) are turned into spaces, and repeated spaces are collapsed into one.

> *All of these rules live in Modules\TitleNormalizer.ps1, in the Normalize-Title function, in case you ever want to adjust or extend any of them.*

7. Title aliases: when the name is genuinely different

The rules in the previous section solve every case where the difference between two names is just a region, language, version, or format tag. But there are cases where the game's name genuinely changes from one region to another — especially with official Spanish translations, or with abbreviated titles — and no automatic rule can guess that it's the same game without risking merging unrelated games by mistake. Some real examples:

Name A	Name B	Why don't they match on their own?
Narnia - El león, la bruja y el armario	The Chronicles of Narnia: The Lion, the Witch and the Wardrobe	Fully translated title
Harry Potter 5 - La Orden del Fénix	Harry Potter and the Order of the Phoenix	Spanish numbering vs. full English name
Golden Sun 2	Golden Sun - La Edad Perdida	Added numbering vs. official subtitle
Zelda - The Minish Cap	The Legend of Zelda, The - The Minish Cap	Abbreviated name vs. full title
Piratas del Caribe - El cofre del hombre muerto	Pirates of the Caribbean - Dead Man's Chest	Translated title

For these cases there's Config\TitleAliases.json: an editable dictionary that maps an already-normalized variant to its canonical form. The program checks it as the last step, right after applying all the rules from section 6 — if the result matches a key in the file, it's replaced with the associated value before grouping.

File format:

```
{
"narnia leon bruja armario": "chronicles of narnia lion witch wardrobe",
"golden sun edad perdida": "golden sun 2 edad perdida",
"piratas del caribe cofre del hombre muerto": "pirates of caribbean dead mans chest"
}
```

How to add a new alias yourself:

1 Note down the two (or more) exact file names that should be considered the same game and aren't.

2 Pick which of the two you want to use as the "canonical form" (the alias value) — it doesn't matter which, it just has to be the same for every variant of that game.

3 Open Config\TitleAliases.json with a text editor (Notepad, Notepad++...).

4 Add a new line with the alternate name already in lowercase and without region/version tags (imagine it has already gone through the rules from section 6), pointing to the canonical name you chose.

5 Save the file. No special restart is needed, the program reads it on every run.

> *If you're not sure how a name will come out after normalizing, that's fine: write the key as close as possible to what you think it'll be (all lowercase, no parentheses/brackets, no accents) and try it — if it doesn't match the first time, run the program and compare it with the normalized name that already worked for the other file in the same group.*

8. Associated files: save files and configurations

Many ROMs have their own files in the same folder, besides the ROM file itself: save files (.sav, .srm, .sram, .state) or per-game controller configuration profiles (for example, Game.dsk.p2k.cfg for Amstrad CPC). When a ROM gets moved to _duplicates for being a duplicate, the program automatically moves these associated files along with it, in the same step — so they never end up orphaned, pointing to a ROM that's no longer in its original folder.

The program recognizes a file as associated with a ROM in two ways:

• Simple match: same file name without the extension ("Game.sav" belongs to "Game.gba").

• Compound extension: the associated file starts with the FULL ROM name, extension included, followed by more extensions of its own ("Barbarian (Europe).dsk.p2k.cfg" belongs to "Barbarian (Europe).dsk").

This feature is controlled by the MoveAssets setting in Config\Settings.ps1 (on by default). One important caveat: the program never treats another real ROM as if it were an associated file, even if it shares the same base name with a different extension — for example, "Game.sfc" and "Game.smc" are two independent copies, each with its own entry in the cleanup plan, not an "associate" of the other.

An associated file is never overwritten

If a file with that name already existed at the destination folder (for example, a save file from a previous cleanup), the program detects it and warns instead of silently overwriting it — the original file that was already there is never lost.

9. Cleaning up orphaned images, videos, and manuals

RetroBat keeps each game's images, videos, and manuals (usually downloaded with ScreenScraper) in the images\, videos\, and manuals\ folders of each system. When you delete a ROM (or CleanROMs moves a duplicate), its image/video/manual is left behind unused — an "orphan" file. This option detects them and moves them (never deletes them, and the move is recorded, so menu option 2 can put them back) to _duplicates\\images, \videos, or \manuals.

How it knows which file belongs to which ROM

ScreenScraper (through RetroBat) adds a suffix to the ROM name depending on the media type, for example: Game-marquee.png, Game-video.mp4, Game-manual.pdf. The program recognizes these suffixes and strips them before comparing, so "2 Game Pack! - Uno & Skip-Bo (Europe)-marquee.png" is correctly recognized as the image for "2 Game Pack! - Uno & Skip-Bo (Europe).zip" — dashes that are already part of the title itself are left untouched; only the suffix is trimmed if it matches exactly at the end.

Currently recognized suffixes: -image, -video, -marquee, -thumb, -wheel, -manual, -boxfront, -boxback, -box2dfront, -box2dback, -box3d, -fanart, -title, -screenshot, -screenshottitle, -cartridge, -support, -mix, -bezel, -map.

> *This cleanup also checks each system's manuals\ folder (it used to only check images\ and videos\). It does not check downloaded_images\, _duplicates\, or folders for unsupported systems (see section 17).*
>
> *Since this version, every file this cleanup moves is recorded in the plan, so "Undo the last cleanup" also brings the images, videos, and manuals back (see section 16).*

10. Requirements and installation

You need PowerShell 7.3 or later (Windows PowerShell 5.1, which ships with Windows by default, is NOT enough): https://github.com/PowerShell/PowerShell/releases

Download the installer for your system (for example, PowerShell-7.x.x-win-x64.msi) and run it with the default options.

Optional: 7-Zip

If you have 7-Zip installed, the program can look inside your .7z files to detect region/language even when those tags are only present on the inner file name, not on the .7z itself. It's found automatically in the PATH, in typical install locations ("C:\Program Files\7-Zip"), and in the Windows registry — no manual setup needed. Without 7-Zip, .7z files are still processed normally, just by their own file name.

Installation

Copy the entire "CleanRoms" folder inside the "roms" folder of your RetroBat installation. Then double-click **"Run CleanROMs.bat"** — this is the recommended way to start it, especially the first time.

That launcher automatically unblocks every file in the folder and starts main.ps1 for you. This avoids a common error the first time you run a script downloaded from the internet: Windows marks files from a downloaded/unzipped folder as "blocked", and PowerShell's default policy then refuses to run main.ps1 directly, showing something like:

> *main.ps1 : File ...main.ps1 is not digitally signed. You cannot run this script on the current system.*

If you'd rather run main.ps1 directly (right-click it and choose "Run with PowerShell 7", or from a terminal with .\main.ps1) and you hit that error, just unblock the folder once and it won't happen again:

> *Get-ChildItem -Path "C:\RetroBat\roms\CleanRoms" -Recurse | Unblock-File*
>
> *If you already had a previous version installed, delete the whole CleanRoms folder before extracting the new one — some unzip tools don't overwrite existing files by default (in particular Config\TitleAliases.json), and you could end up with a mix of versions that doesn't include the latest aliases or settings.*

11. First run: RetroBat, language, and ROM priority

The first time you run the program (or if you delete Config\UserSettings.json) it will ask you three things, one after another:

1. Interface language

> *Selecciona idioma / Select language:*
> *1) Español*
> *2) English*

This is saved so it won't be asked again, and from here on the rest of the first-run questions are shown in the language you chose.

2. ROM priority (Spanish or English)

> *Which ROMs do you want the program to prefer when two versions of the same game compete? 1) Spanish (prioritizes Spain > Europe > USA > Japan)*
> *2) English (prioritizes USA > Europe > Spain > Japan)*

This is a **completely separate** setting from the interface language — it decides which region/language scoring table is used (see section 2), not what language the menus are shown in. You can perfectly well have the menus in English and still want to keep Spanish ROMs when there's a choice, or the other way around.

3. RetroBat path

> *RetroBat path not configured yet.*
> *Path to RetroBat's 'roms' folder [C:\RetroBat\roms]:*

Press Enter to accept the suggested path (the folder that contains CleanRoms), or type another one.

All three answers are saved to Config\UserSettings.json so they won't be asked again. If you move RetroBat elsewhere, or simply want to re-pick any of the three, you can delete that whole file, or change just what you need from menu option 5 without touching anything else (see sections 13 and 19).

12. Configuring the program in depth

Almost all of the program's behavior is controlled from files inside the Config\ folder, editable with any text editor:

Config\UserSettings.json

Stores the RetroBat path, the interface language, and the ROM priority you set on first run (section 11). Delete it if you need it to ask you everything again, or change just what you want from menu option 5.

Config\Settings.ps1

Almost everything else lives here. The settings most relevant day-to-day:

Setting	What it controls
PreviewOnly	If $true, never actually moves/deletes anything even if you confirm with "Y" (see section 14)
MoveAssets	If $true (default), moves associated files along with each ROM (section 8)
RemoveDuplicates	Reserved: would allow deleting duplicates instead of moving them — off by default. The only thing the program ever deletes is the purge of _duplicates (section 27)
UndoHistoryLimit	How many past cleanups can be undone, besides the most recent one (default 10, section 16)
MergeGamelistMetadata	If $true (default), carries favorites and play counts from a moved duplicate over to the ROM that stays (section 25)
HashCache	If $true (default), reuses the hashes of files that haven't changed since the previous run (section 26)
PurgeDuplicatesAfterDays	Days a copy must have been in _duplicates before the purge deletes it (default 90, section 27)
HashParallelThreshold / HashParallelism	From how many files onward the hash is calculated in parallel, and how many at a time (section 15)
RomExtensions (Extensions.json)	List of 110+ file extensions recognized as ROMs (section 28)
IgnoredFolders (Extensions.json)	Folders that are never scanned for ROMs (images, videos, manuals, _duplicates...) (section 28)
Systems.json	List of 60+ supported systems and their folder name (section 28)
$Global:OrphanedMediaFolders	Which subfolders the orphan cleanup checks (images, videos, manuals)
$Global:MediaFileSuffixes	Recognized image/video/manual suffixes (section 9)

Config\DecisionWeights.ps1

Contains the two full scoring tables from section 2 (Spanish priority and English priority), plus the rest of the tables (dump quality, status, hacks, version, revision). Feel free to edit them if, for example, you want to change exactly what the two priorities differ on, or penalize prototypes less.

Config\TitleAliases.json

The title aliases from section 7 — for games whose name genuinely changes between languages or editions.

13. The main menu (the 7 options)

On startup, the program asks you what you want to do:

What do you want to do?
1) Clean up duplicate ROMs
2) Undo the last cleanup
3) Clean up orphaned images/videos/manuals
4) ALL: Move ROMs and images/videos/manuals for every system
5) Configuration (RetroBat path / language / ROM priority)
6) Purge _duplicates (DELETE old copies)
7) Exit

Option 1 — Clean up duplicate ROMs

It asks you for the system (or "0) ALL SYSTEMS"). The system menu only shows systems that actually have ROMs in their folder — if a system has no valid file at all, it doesn't even appear in the list. It then scans, groups, scores, shows you the full plan, exports the reports (section 15), and only moves files if you answer "Y" to the confirmation. It also organizes any loose hacks it finds: first it moves any loose ROM whose name indicates it's a hack/beta/demo/etc. (see section 5) into "# Hacks y Otros #", then it looks inside that folder for byte-identical copies (same hash) to deduplicate those too.

Option 2 — Undo the last cleanup

Puts back in its original place any file that was actually moved — checking on disk, not just on paper, so if that time you only previewed or cancelled, it will simply tell you there's nothing to undo. It can undo more than just the most recent session — see the full detail in section 16.

Option 3 — Clean up orphaned images/videos/manuals

See section 9 for the full detail.

Option 4 — ALL: clean up everything, every system

Chains option 1 and option 3, but directly across every configured system, without asking you which one again. Each step has its own preview and its own separate Y/N confirmation.

Option 5 — Configuration

Shows the currently configured RetroBat path, interface language, and ROM priority, and lets you change any of the three without having to delete Config\UserSettings.json by hand (see sections 11 and 19):

== Configuration ==
Current RetroBat path : C:\RetroBat\roms
Current interface language : en
Current ROM priority : Spanish

1) Change the RetroBat path
2) Change the interface language
3) Change the ROM priority (Spanish/English)
4) Change all of the above
0) Back to the main menu

After changing what you need, it goes straight back to the main menu.

Option 6 — Purge _duplicates (DELETE old copies)

The only option that deletes files for real. It deletes the copies that have been in _duplicates for N days or more, after showing you the list and asking for confirmation. See section 27 for the full detail.

Option 7 — Exit

Closes the program.

14. Simulation mode (PreviewOnly)

In Config\Settings.ps1 you can turn on:

```
PreviewOnly = $true
```

With this, even if you confirm with "Y", the program only prints what it would do ([PREVIEW MOVE]) without actually moving anything. It's respected **everywhere** in the program: the regular ROM cleanup, the orphaned images/videos/manuals cleanup, both steps of organizing/deduplicating hacks, the gamelist.xml merge (section 25), and the purge of _duplicates (section 27). Highly recommended the first time you try it on a new collection, or after changing an important setting.

It can also be turned on for just a single run using the -PreviewOnly command-line parameter, without having to edit the file (see section 21).

15. Generated reports and the full log

Every time a cleanup plan is generated (even if it finds no duplicates), three files are exported to Resultado\:

• CleanPlan.json — the full plan detail (also used by "Undo the last cleanup").

• CleanPlan.csv — summarized version, to open in Excel.

• CleanPlan.html — visual report with stats and a color-coded table, to open by double-clicking in your browser.

Depending on what you use, Resultado\ may also contain:

• HashCache.tsv — the hash cache (section 26); safe to delete.

• GamelistBackups\ — a copy of each gamelist.xml before it is modified (section 25).

• PurgeLog_<date>.txt — the list of what each purge deleted (section 27).

• DuplicatesSeen.tsv — when the purge first saw files with no known move date (section 27).

• History\ — the archived sessions that "Undo the last cleanup" can go back to (section 16).

In addition, every full session (everything you see on screen, including your answers to the menu and Y/N confirmations) is saved to a text file inside Logs\CleanROMs_.log — useful for reviewing exactly what happened after closing the console, or for attaching it if you report an issue (see section 30).

Parallel hashing for large collections

Hashing (used to find exact duplicate hacks, and to verify moves) runs in parallel on large batches instead of one file at a time, using PowerShell 7's ForEach-Object -Parallel. Controlled by HashParallelThreshold and HashParallelism in Config\Settings.ps1; by default it's not worth parallelizing below 20 files, and it uses 4 at a time above that. Lower HashParallelism to 1 if you're on a slow HDD and notice things got slower instead of faster.

16. Undoing the last cleanup

Expanding on menu option 2: for every file the plan moved, the program checks whether it's really in its destination folder before restoring it, and if the original spot is already occupied by something else, it warns you and skips it instead of overwriting. At the end it gives you a summary: how many were restored, how many didn't need touching, and how many were skipped due to a conflict.

This option also restores associated files (save files, controller configs...) that were moved alongside a ROM, not just the ROM itself — and it covers moves made while organizing loose hacks into "# Hacks y Otros #" and deduplicating them by hash (see section 5) too, not just the regular ROM cleanup.

More than one session to undo

Every time you start a new cleanup, the previous session is automatically archived to Resultado\History\ before it begins. If there's any archived session, option 2 lets you pick which one to undo:

> *Which cleanup do you want to undo?*
> *0) The most recent one (Enter)*
>
> *1) 2026-08-20 15:32*
> *2) 2026-08-19 10:05*
> *Pick a number:*

Pressing Enter without typing anything undoes the most recent one, exactly as before — if you never care about this, nothing changes in how you use the program. By default the last 10 sessions are kept (adjustable with UndoHistoryLimit in Config\Settings.ps1); older ones are dropped automatically.

Images, videos, and manuals

Option 2 also brings back the orphaned images, videos, and manuals moved by the orphan cleanup (section 9), including the ones in subfolders such as images\sub\. Every media file that is moved is recorded in the plan like any other move, so after "ALL" followed by "Undo" you get back the ROMs and their images and videos, not just the ROMs. (Before this version those files were moved without being recorded, so Undo could not return them.)

Chained moves: the order matters

Moves are undone from last to first. This matters when the same file was moved twice in one session — for example a loose hack that is first organized into "# Hacks y Otros #" and then, being an exact copy of another, sent to _duplicates. Undoing from last to first brings it all the way back to its original folder, instead of leaving it half-way in "# Hacks y Otros #".

If a file can't be restored

A file that cannot be restored (locked by another program, no permission...) is reported and counted ("Failed: N") but does not stop the rest. In unattended mode the exit code is then 2 (section 21). Files that were deleted by a purge (section 27) are simply skipped, and everything else is restored.

17. Supported systems and extensions

The program currently recognizes 60+ systems (Nintendo, Sega, Sony, arcade, home computers...) and 110+ different file extensions. The full list is in Config\Systems.json and Config\Extensions.json (section 28) and can be freely extended. Some examples:

Category	Examples
Nintendo	NES, SNES, N64, GameCube, Wii, Game Boy (Color/Advance), NDS
Sega	Master System, Mega Drive/Genesis, Game Gear, Saturn, Dreamcast
Sony	PlayStation, PlayStation 2, PSP, PS Vita (.vpk)
Arcade	MAME, FinalBurn Neo, Neo Geo, CPS1, CPS2, CPS3
Home computers	Amstrad CPC, ZX Spectrum, MSX, Commodore 64, X68000

Some systems in the list have limited support because their games come as a "folder" rather than a single file (PS3, PS Vita without .vpk, PC DOS, Nintendo Switch in some cases) — the program detects files by extension, not whole folders, so those formats won't be recognized even though the system is in the list.

> *Multi-core arcade systems (MAME, FinalBurn Neo, CPS1/2/3, Neo Geo, Cave, Zinc, Namco System 11/22...) are each processed independently, without comparing ROMs across different systems — this is intentional: RetroBat sometimes needs a copy of the same arcade file in several different system folders so that each core can launch it, so they are never treated as "duplicates" of each other even though they may look alike. The same applies, for example, between "megadrive" and "sega32x", or between "nes" and "nes3d": they're different systems as far as RetroBat is concerned, and the program never compares ROMs from one system against another.*

18. Special file names (brackets, apostrophes, underscores)

ROM names frequently use characters that, in PowerShell, have a special meaning if not handled carefully — especially brackets ([!], [E], [T+Eng]...), which PowerShell can interpret as search wildcards instead of literal text. The program internally uses "literal" paths in every file operation (checking existence, moving, deleting, undoing...) precisely so this is never an issue: a ROM with brackets in its name is processed exactly the same as one without them.

This matters because, without this care, a ROM with brackets in its name could silently disappear from the scan (the program would think the file doesn't exist), which in turn would prevent detecting its real duplicates. If you ever see a "File does not exist" warning in the log for a ROM you know is really in its folder, please report it (section 30) along with the exact file name.

Underscores

As already explained in section 6, some sets (mostly SNES) use underscores instead of spaces/parentheses. Detection of Hack, Translation, the Spain region, and version/revision also recognizes this style: for example, "Street_Fighter_5_Hack_.smc" is detected as a hack just like "Street Fighter 5 (Hack).smc", because the program treats an underscore as a valid word separator in these checks, not as if it were part of the word.

19. Interface language and ROM priority

The program has **two** completely independent language-related settings, which are worth not confusing:

Setting	What it controls	Where to change it
Interface language	What language the menus, messages, and confirmations are shown in on screen	Menu option 5, sub-option 2
ROM priority	Which region/language scoring table is used (section 2) — decides which copy wins when two versions compete	Menu option 5, sub-option 3

Both are asked separately on first run (section 11), and both can be changed at any time without affecting the other. You can, for example, have the program in English (English menus) and still want the Spanish copy to win when there are two versions of the same game — or the exact opposite.

Interface language

Available in Spanish and English. The answer is saved in Config\UserSettings.json and won't be asked again. The translation covers the program's whole usual flow: menus, preview, confirmations, final summary, orphan cleanup, undo, and the most important notices.

ROM priority

Available in Spanish and English. Determines which of the two Config\DecisionWeights.ps1 tables is active ($Global:DecisionWeights_ES or $Global:DecisionWeights_EN). It doesn't affect anything else — not the menus, not which files are considered hacks/betas/etc, and not any other scoring criterion (dump quality, version, revision...), only the region and language table.

20. About renaming ROMs (RENAME)

You may see "RENAME" mentioned in the cleanup summary, alongside KEEP, MOVE, and DELETE. To be clear: the program never renames any ROM. That counter always shows 0.

Name normalization (section 6) — stripping dashes, parentheses, region tags... — is used purely internally, to decide which copies are the same game. It's never applied to the actual file name: the ROM that's kept stays with exactly the name it already had.

This is intentional: renaming the file RetroBat already has indexed would break its association with the images, videos, and manuals already downloaded via ScreenScraper, since gamelist.xml links them by the exact file name. Changing the name without also updating gamelist.xml would make you lose that already-obtained artwork for the kept game.

21. Command-line use: automated/unattended runs

For unattended runs (for example, with Windows Task Scheduler), main.ps1 accepts parameters that skip the menu entirely. Without them, the program behaves exactly as it always has — none of this is required or changes normal use.

Parameter	What it does
-Action Clean|Orphans|All|Undo|Purge	What to do: Clean = clean up duplicate ROMs (and organize hacks), Orphans = orphaned media, All = everything, Undo = undo the last cleanup, Purge = delete the copies that have been in _duplicates for a long time (section 27)
-System <folder>	The system's folder name (e.g. "snes", "gba"); omit or use "ALL" for every system. Ignored with -Action Undo and -Action Purge
-OlderThanDays <n>	Only with -Action Purge: delete the copies that have been in _duplicates for this many days or more (1 to 3650). If omitted, PurgeDuplicatesAfterDays from Config\Settings.ps1 is used (90 by default)
-Yes	Auto-confirms every Y/N prompt instead of waiting for someone to answer — this is what actually makes a run unattended. It also makes the first run (language, ROM priority, RetroBat path) pick default values without asking
-PreviewOnly	Forces simulation mode (section 14) for this run only, without having to edit Config\Settings.ps1

Examples:

```
pwsh -File main.ps1 -Action Clean -System snes -Yes
pwsh -File main.ps1 -Action All -Yes
pwsh -File main.ps1 -Action Undo -Yes
pwsh -File main.ps1 -Action Purge -OlderThanDays 120 -Yes
```

The -System parameter takes the folder name exactly as it appears under RetroBat\roms (for example "snes"), not the long descriptive name shown in the interactive menu — that's what anyone setting up a scheduled task will already have at hand.

> *Without -Action, main.ps1 behaves exactly as it always has (interactive menu). Without -Yes, -Action still skips the menu but still asks for confirmation at each step, same as interactive mode — -Yes is specifically what keeps the program from sitting there waiting for an answer nobody's going to give. If it's the very first run (no Config\UserSettings.json yet) and -Yes can't determine a valid RetroBat path, the program stops with a clear error instead of waiting forever.*

Exit codes

Code	Meaning
0	Everything went fine.
1	Unexpected error (details in crash_log.txt) or invalid arguments, such as an unknown system.
2	The run finished, but one or more actions failed (a file that couldn't be moved, restored, or deleted...). See the log.

In unattended mode (-Yes) the program never waits for a key press at the end, so a scheduled task always finishes on its own, and Task Scheduler can tell from the exit code whether something needs your attention.

22. Tests performed

Before considering each fix closed, it has been tested with real data, not just isolated cases. The process followed in each round of testing has been:

1 First, on small or medium system folders (GBA, SNES, Amstrad CPC), using the complete listing of real files in those folders (not a sample), to find and fix name patterns the rules didn't yet recognize.

2 Then, on the total ROM count of a full collection (35+ systems at once, over 13,600 ROMs in total), checking the resulting log for how many files are processed, how many duplicate groups are detected, and how many actually end up being moved, system by system.

3 Finally, reviewing the full log file for errors or warnings (there should be none), and comparing the file listing before and after cleanup — with the MD5 hash of every file — to confirm that no image, video, or manual belonging to a kept ROM is moved by mistake, that no ROM is moved without genuinely being a duplicate, and that everything organized into "# Hacks y Otros #" really corresponds to an actual hack, beta, demo, prototype, translation, homebrew, or pirate copy.

This last kind of check (comparing full listings with MD5 hashes before and after, on a real collection of over 94,000 files) uncovered several bugs that no amount of testing with isolated cases had caught — among them, recognizing "Hack"/"Translation"/the Spain region with underscores (section 18), and the false positives from Beta/Demo/Prototype/Pirate matching substrings inside other words (section 5) — precisely by comparing expected behavior against what actually showed up in the log and the file listing, not just reviewing the code on paper.

Automated tests

Besides the manual rounds described above, the project now has an automated test suite that anyone can run: Tests\Run-AllTests.ps1 (or Run-AllTests.bat) runs more than 750 checks, each one with real files in a temporary folder, and ends with exit code 1 if any test fails or is interrupted by an error.

Two more scripts complete it. Tests\Check-Project.ps1 checks the syntax of every .ps1 file, that every file the program loads exists with the exact capitalization (it matters on Linux), and that the .json files are valid. Tests\Smoke-Unattended.ps1 runs the real program in unattended mode (-Action ... -Yes) on a small sample collection and checks that it finishes by itself with the right exit code.

A GitHub Actions workflow (.github\workflows\tests.yml) runs all three on every push and pull request, on Windows and on Linux. Publishing a version is automated too (section 29).

These tests have already caught real problems, such as multi-file games being split apart (section 23), "Undo" leaving hacks half-way (section 16), and orphaned media that could not be undone.

23. Multi-file games (.cue/.bin, .gdi, multi-disc, .m3u)

Disc-based games (PlayStation, Saturn, Dreamcast, Sega CD, PC Engine CD...) are made of several files: a .cue (or .gdi) that describes the tracks, one or more .bin/.raw files with the data, and often an .m3u playlist for games with several discs. Until now each file was treated as a separate ROM, which broke games: a .cue and its .bin looked like copies of each other and one of them was moved, and the discs of a game looked like duplicates of each other. Now each game is handled as a single unit.

How a game is put together

• A .cue, .gdi, or .m3u takes along the files it points to: the track files (.bin, .raw...), the discs listed in an .m3u, and the tracks of each disc. Those files stop being treated as ROMs on their own.

• Discs of the same game without an .m3u ("(Disc 1)", "(Disc 2)", "(Disk 1)", "(CD 1)", "(Disc 1 of 2)") form a set, represented by the lowest-numbered disc.

• The program reads the .cue/.gdi/.m3u to know which files belong together. File names with different capitalization, and Windows-style folder separators inside the file, are understood.

What changes when it cleans

• A game is kept or moved WHOLE, never half: you will never end up with a .cue and no .bin. A USA set and a Europe set are compared as sets, using the representative file.

• If any file of a game would clash with a file already in _duplicates (or with another game in the same plan), that game is left exactly where it is and a warning is shown.

• Every file is its own move in the plan, so "Undo the last cleanup" puts each one back in its exact original folder, subfolders included.

• A file referenced by more than one .cue/.m3u (for example a shared track) is never moved, so the copy you keep keeps working.

• The screen tells you how many multi-file games were handled as a single unit.

> *Known limits: .ccd/.mds descriptors are not interpreted, and the organizer of loose hacks (section 5) still works file by file.*

24. Manual decisions that stick (Config\Overrides.json)

If you disagree with what the scoring system picks for a particular game, you can say so once and it will be respected in every later cleanup, including scheduled (-Yes) ones. Create Config\Overrides.json (there is an example in Config\Overrides.example.json):

```
{
  "Prefer": [ { "System": "snes", "File": "Super Mario World (Europe).sfc" } ],
  "Ignore": [ { "System": "snes", "Title": "Super Mario World" },
              { "System": "nes",  "File": "Zelda II (USA).nes" } ]
}
```

What each list does

• Prefer: that ROM always wins its duplicate group; the other copies are moved as usual. The reason shown in the preview says it was chosen manually.

• Ignore: the group is left completely untouched (nothing is kept or moved). Identify it by "Title" (the name printed after "Group:" during a cleanup, or the normal title of the game) or by "File" (any copy in the group with that file name).

• "System" is the system's folder name (snes, nes...). Leave it out, or use "*", to apply to every system. Names are not case-sensitive.

Remembering your tie-break answers

When two copies tie and the program asks which one to keep (section 3), it now offers to remember your answer for future cleanups. If you accept, it is saved to Overrides.json as a "Prefer". With -Yes it never asks and never saves anything.

> *If Overrides.json doesn't exist, nothing changes. If it cannot be read (for example a typo in the JSON), you get a warning and it is ignored — and the program never overwrites a file it can't read, so what you wrote by hand is never lost.*

25. RetroBat's gamelist.xml (favorites, play counts...)

RetroBat keeps your favorites, play counts, last-played dates, and ratings in each system's gamelist.xml. When a duplicate ROM is moved, its data in the gamelist.xml is carried over to the ROM that stays, so you don't lose your favorites or play history. This also applies to the exact duplicates found inside "# Hacks y Otros #".

Field	What happens
favorite	Kept if either copy was a favorite
playcount, gametime	The higher of the two (they are NOT added up, so undoing and cleaning again never inflates them)
lastplayed	The most recent date
rating	The highest rating
name, desc, developer, publisher, genre, players, releasedate...	Copied only if the ROM that stays doesn't have them
image, video, and manual paths	Never copied (they belong to the moved file)

• No entry is ever deleted, so "Undo the last cleanup" keeps working without touching the gamelist.xml. If the ROM that stays has no entry, a new one is created.

• Before saving, a copy of the gamelist.xml is stored in Resultado\GamelistBackups\ (the last UndoHistoryLimit copies per system).

• With PreviewOnly it only reports what it would do. To turn all of this off, set MergeGamelistMetadata = $false in Config\Settings.ps1 (it disables every change to the gamelist.xml).

Entries follow the files when they change folder

Moving a file also changes its path. When the hack organizer (section 5) moves loose hacks into "# Hacks y Otros #", the gamelist.xml entry of each hack follows the file: its path is updated (with a backup first), so RetroBat does not lose that hack's favorites and play history. "Undo the last cleanup" puts the path back. Because the entries are found at their new paths, exact duplicates detected in the same pass also merge their data, as described above. Only entries that exist are changed; a hack without an entry is moved as usual and the gamelist.xml is left untouched.

26. Hash cache and faster deduplication

Hashing thousands of ROMs is the slowest part of a cleanup, and every run used to read and hash the same unchanged files again. The program now keeps Resultado\HashCache.tsv with each file's hash plus its size and last modification date at the time it was computed. A stored hash is reused only while both are exactly the same; if either changes, the file is hashed again.

• The next cleanups skip every file that hasn't changed, and the screen tells you how many hashes were reused.

• If you change HashAlgorithm (MD5, SHA256...), the cache is discarded.

• It is only an accelerator: if it is missing, damaged, or cannot be written, everything works the same, just slower. Delete Resultado\HashCache.tsv to start over, or set HashCache = $false in Config\Settings.ps1 to turn it off. (The only way it could ever be wrong is a file whose content changes without its size or date changing, which practically never happens.)

Skipping files that can't have a duplicate

When deduplicating "# Hacks y Otros #" (section 5), two files can only be identical if they weigh exactly the same. Files whose size is unique in the folder are no longer read or hashed at all, which in a large hacks folder means most files are never touched.

27. Purging _duplicates (menu option 6)

_duplicates only ever grows. Option 6 deletes the copies that have been there for N days or more (PurgeDuplicatesAfterDays in Config\Settings.ps1, 90 by default; the menu asks for the number and pressing Enter keeps the default). It is the only thing in the whole program that deletes files for real, so it is deliberately careful:

• Age counts from when the file was MOVED to _duplicates (read from the plans saved in Resultado\), not from the file's own date, which is the date of the original ROM. If a file was moved several times, the last move counts. Associated files (save files...) and media in subfolders count from the date of their move.

• Files with no known move date (for example from plans that have already left the history) are never deleted all at once: their count starts the first time the purge sees them (Resultado\DuplicatesSeen.tsv), and they only become candidates N days later. If such a file changes, its count starts over.

• It only touches files inside _duplicates, removes the folders left empty, and writes what it deleted to Resultado\PurgeLog_<date>.txt.

• With PreviewOnly it only simulates. A file that cannot be deleted is reported without stopping the others (exit code 2 in unattended mode).

What it shows you before deleting

First a count and size per system, then the oldest files, then the total, and then it asks for confirmation:

```
PURGE OF _duplicates: copies moved 90 days ago or more
  snes: 12 file(s), 48.3 MB
  gba: 3 file(s), 21.0 MB
Total to delete: 15 file(s), 69.3 MB
15 file(s) (69.3 MB) will be PERMANENTLY DELETED.
This CANNOT be undone. Continue? (Y/N)
```

From the command line

```
pwsh -File main.ps1 -Action Purge -OlderThanDays 120 -Yes
```

With -Yes it does not ask for confirmation, so try it first with -PreviewOnly. After a purge, "Undo the last cleanup" can no longer restore the deleted files: it skips them and restores everything else.

28. Systems and extensions in JSON files

The systems the program knows, the file extensions it treats as ROMs, and the folders it never scans used to be lists inside Config\Settings.ps1, mixed with code. They now live in two JSON files that are safe to edit with any text editor:

File	What it contains
Config\Systems.json	The supported systems as "Name": "folder", grouped by category (desktop consoles, handhelds, home computers, arcade). Entries outside any category also work.
Config\Extensions.json	"RomExtensions": the extensions treated as ROMs, and "IgnoredFolders": folders that are never scanned (images, videos, _duplicates...).

Adding your own without touching those files

Create Config\Systems.custom.json and/or Config\Extensions.custom.json. Both are optional, use the same format, and are added on top of the main files, so updating the program never overwrites your additions. In systems, a repeated name takes the folder from the custom file; in extensions and ignored folders, the lists are simply added together.

```
Config\Systems.custom.json:
{ "Mine": { "My system": "mysystem" } }

Config\Extensions.custom.json:
{ "RomExtensions": [".xyz"], "IgnoredFolders": ["my_stuff"] }
```

If something is wrong

• A single invalid entry (an empty name, a folder name with characters that are not allowed, something that is not text...) is reported with a warning that names the file, and ignored. The rest keeps working.

• If a file cannot be read (for example a comma missing in the JSON) or defines no systems or no extensions, the program stops with a message that names the file and the error, instead of failing later in a confusing way.

• Extensions are normalized: ".SFC" and "sfc" both become ".sfc", and repeated ones are removed.

29. Version number and releases

The version number lives in a single file, VERSION, next to main.ps1. It used to be repeated in the header of every file, in the menu banner, in the log, and in the plans. It is now read at startup and shown in the menu banner, at the top of the log, and in each plan (CleanPlan.json). To publish a new version, that is the only file you change.

Publishing a release on GitHub

The project includes the workflow .github\workflows\release.yml. To publish version 2.8, for example:

```
(1) Put 2.8 in the VERSION file and commit it
(2) git tag v2.8
(3) git push origin v2.8
```

The workflow then checks that the tag (v2.8) matches the VERSION file, runs the checks and the whole test suite on Windows, builds CleanRoms-v2.8.zip together with its SHA256 file, and publishes a GitHub Release with both attached. If the tag and VERSION do not match, nothing is published.

What is inside the ZIP

• Everything goes inside a single folder named with the version (CleanRoms-v2.8). Unzipping it over an older installation therefore never mixes old and new files: unzip it on its own and copy over what you want to keep.

• Nothing personal is included: Config\UserSettings.json, Config\Overrides.json, your .custom.json files, Logs, Resultado and crash_log.txt are never in the ZIP, so installing a new version cannot overwrite your settings or your decisions.

Building the ZIP yourself

```
pwsh -File Tools\Build-Release.ps1
```

It leaves CleanRoms-v<version>.zip and its .sha256 in the dist folder.

Messages and languages

Every message shown on screen, including progress bars and the unexpected-error message, now comes from Config\Strings.ps1 in Spanish and English. An automated test checks that every message the code uses exists in both languages.

30. Reporting issues

If you find a bug, or want a feature added or removed, please email as much detail as you can (what you did, what you expected to happen, and what happened instead) to:

✉ *batserra@gmail.com*

Project repository: https://github.com/batserra/CleanRoms

If the issue involves ROMs not grouping as expected, or files disappearing from the folder without explanation, please attach if you can: the log file from that run (Logs\CleanROMs_.log) and, if possible, a listing of the affected files (for example with dir /s /b > listing.txt from that system's folder). The more real data, the faster the exact cause can be found.